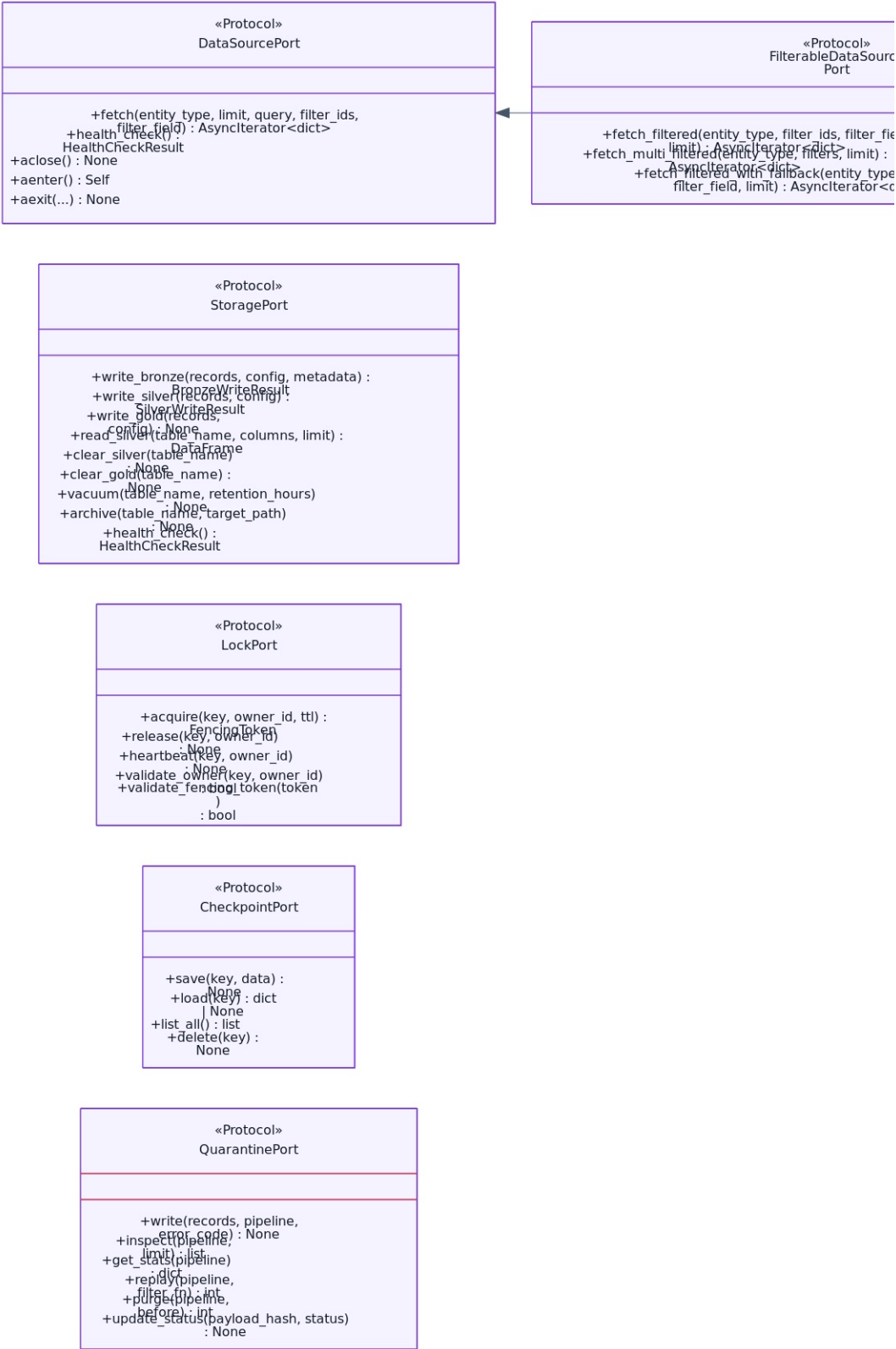


BioETL Class Diagrams with Descriptions

Сгенерировано: 2026-03-02T15:08:12.775826+03:00

01-domain-ports — Domain Ports



«Protocol» AuditPort
+log_write(entry: AuditEntry) : None +get_entries(messages: list<AuditEntry>) : list<AuditEntry>

«Protocol» LoggerPort
+bind(**kwargs) : None +info(msg, **fields) : None +warning(msg, **fields) : None +error(msg, **fields) : None +debug(msg, **fields) : None +exception(msg, **fields) : None

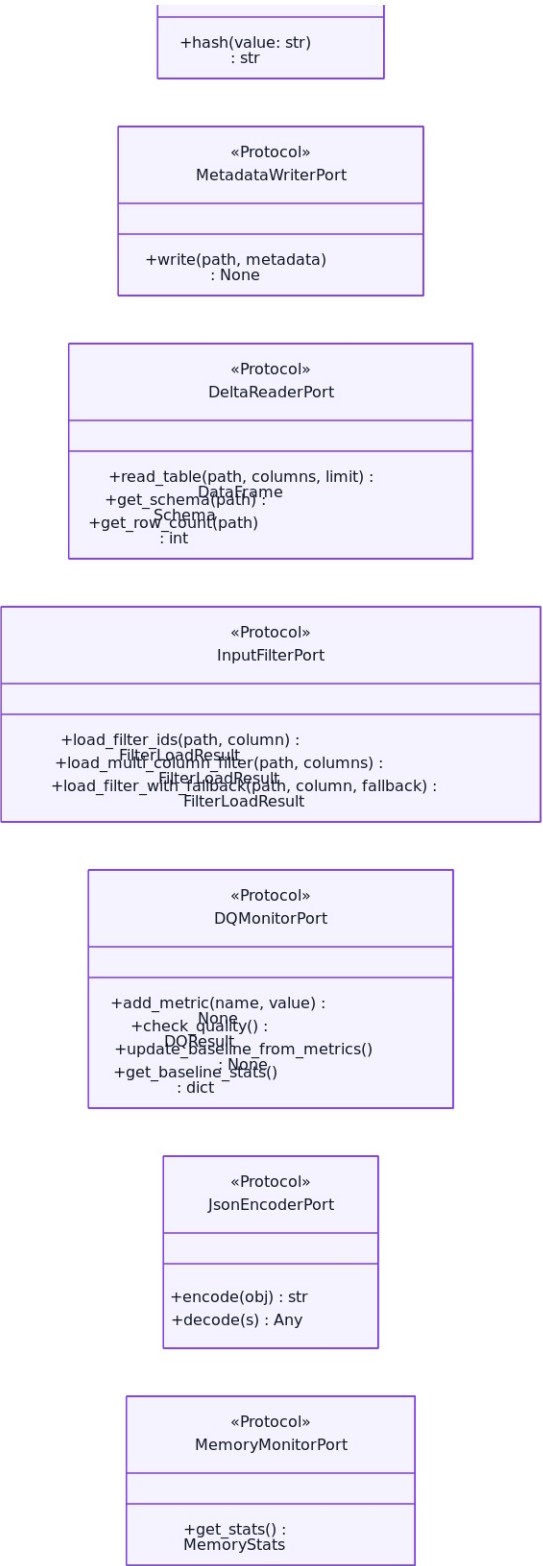
«Protocol» MetricsPort
+observe_histogram(name, value, labels) : None +increment_counter(name, labels) : None +set_gauge(name, value, labels) : None +close() : None

«Protocol» TracingPort
+get_tracer(name) : Tracer +close() : None

«Protocol» CircuitBreakerPort
+get_state() : CircuitBreakerState +get_failure_rate() : float +call(fn) : int +reset() : None

«Protocol» RateLimiterPort
+acquire(tokens) : None +try_acquire() : bool +available_tokens() : int

«Protocol» PiiHasherPort

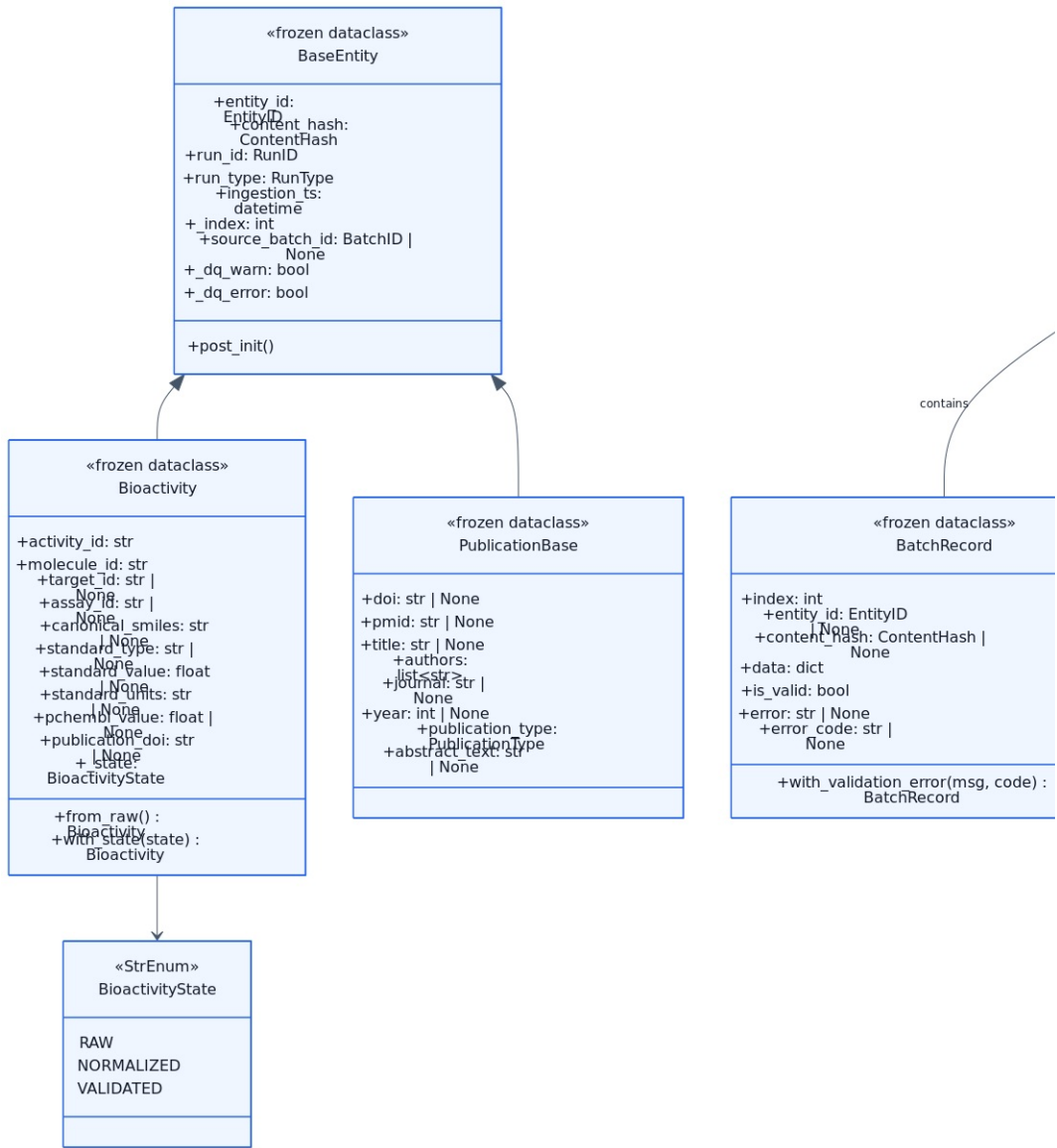


01-domain-ports

Диаграмма Class Diagram: Domain Port Protocols из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 01-domain-ports. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: All Protocol interfaces defined in domain/ports/. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: DataSourcePort, FilterableDataSourcePort, StoragePort, LockPort, CheckpointPort, QuarantinePort. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более

узкие представления при росте сложности.

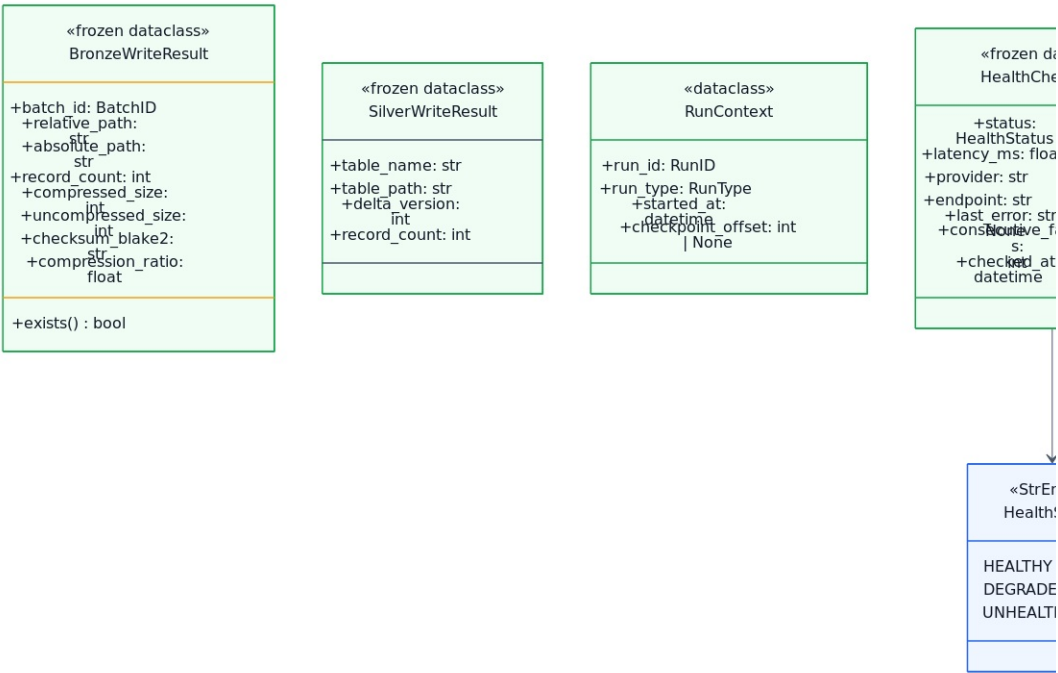
02-entities-aggregates — Entities Aggregates



02-entities-aggregates

Диаграмма Class Diagram: Entities & Aggregates из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 02-entities-aggregates. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Domain entities, aggregate roots, and their relationships.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: BaseEntity, Bioactivity, BioactivityState, PublicationBase, Batch, BatchRecord. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

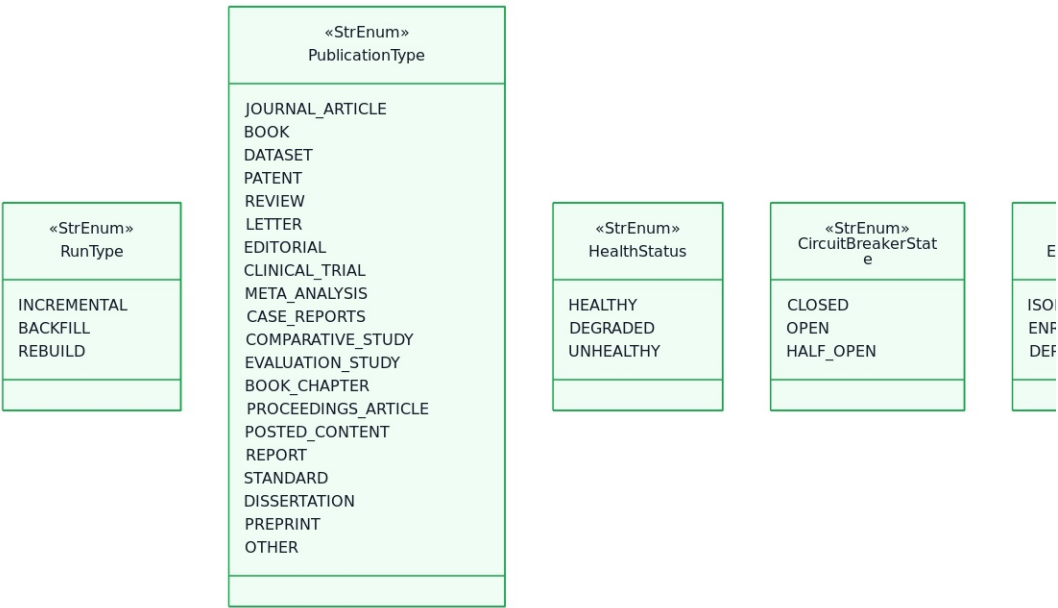
03-value-objects — Value Objects



03-value-objects

Диаграмма Class Diagram: Value Objects из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 03-value-objects. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Immutable domain value objects.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: BronzeWriteResult, SilverWriteResult, RunContext, HealthCheckResult, FencingToken, LockContext. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

04-types-enums — Types Enums



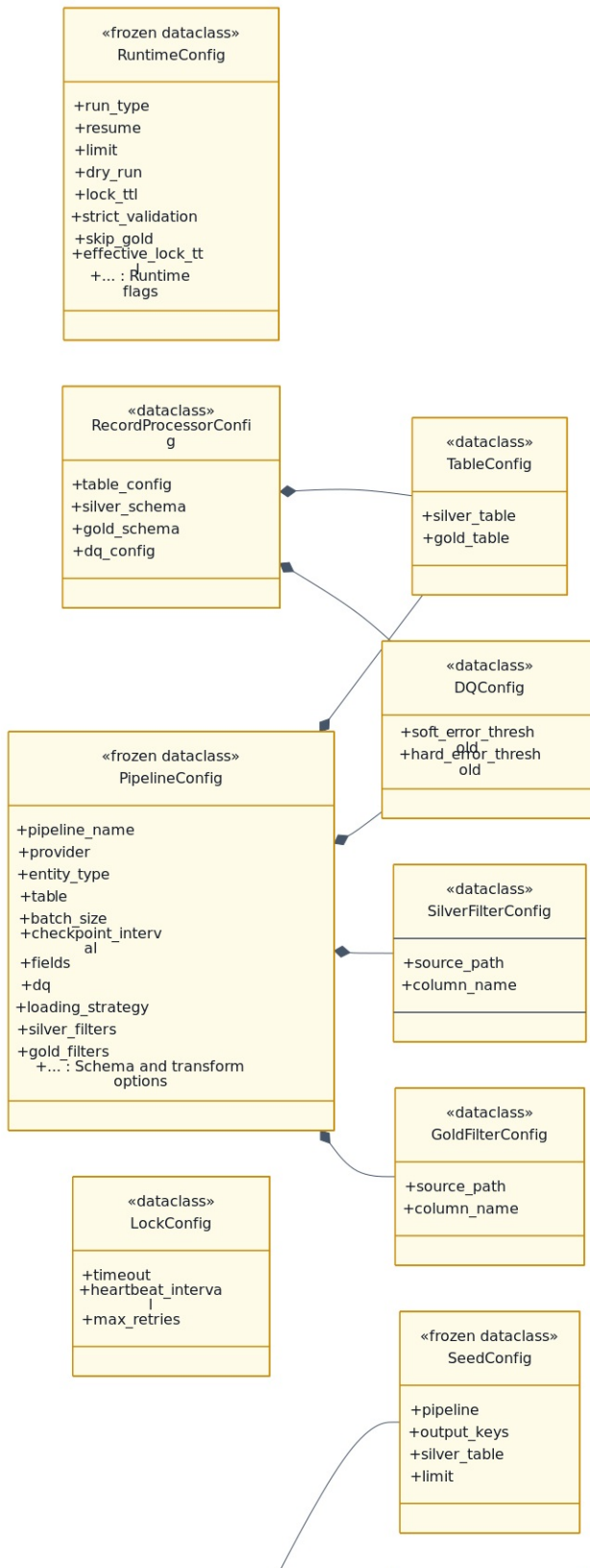
04-types-enums

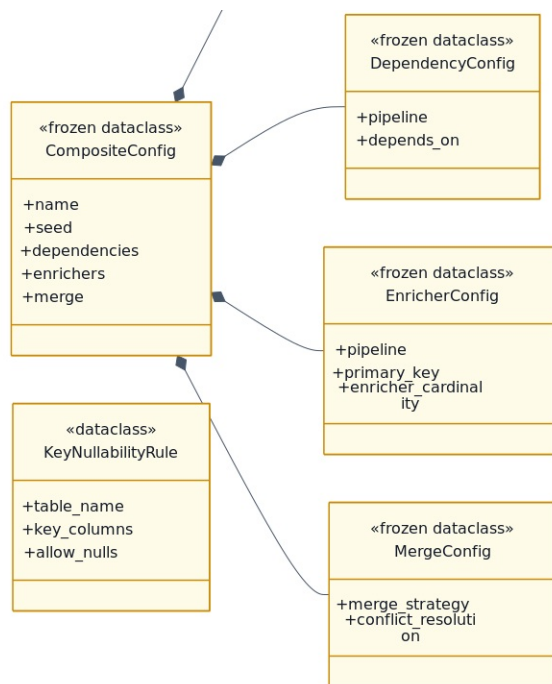
Диаграмма Class Diagram: Types & Enums из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 04-types-enums. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: All type aliases, NewTypes, and enumerations.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: RunID, EntityID, ContentHash, BatchID, RunType, PublicationType. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

05-exceptions — Exceptions

CriticalError, RecoverableError, DataQualityError, ValidationError, SchemaViolationError. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

06-config-classes — Config Classes

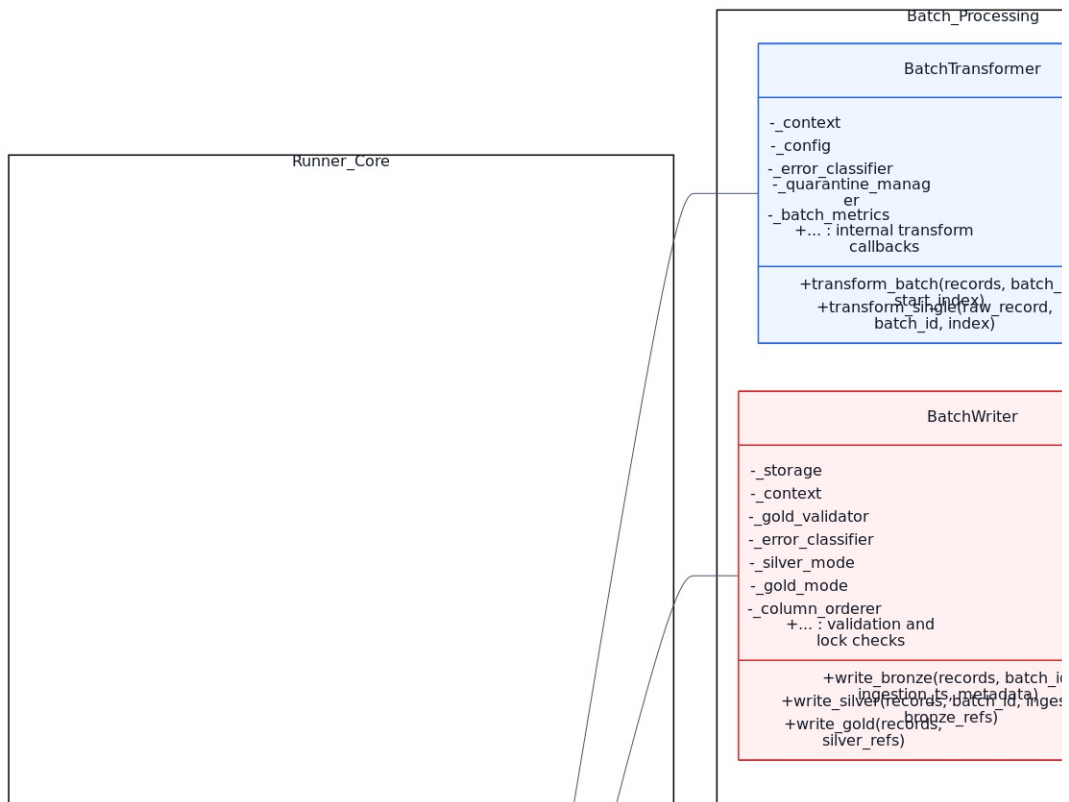


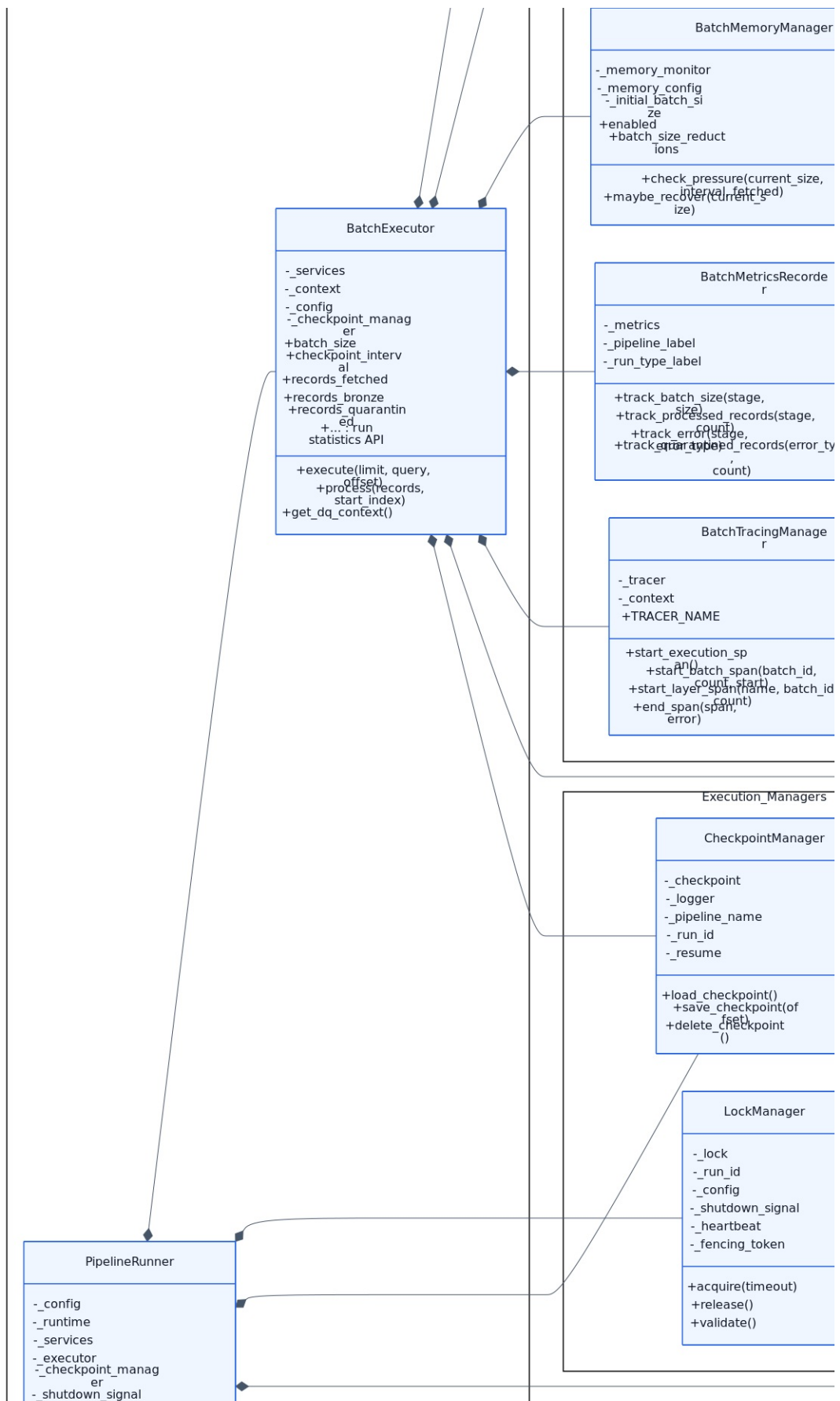


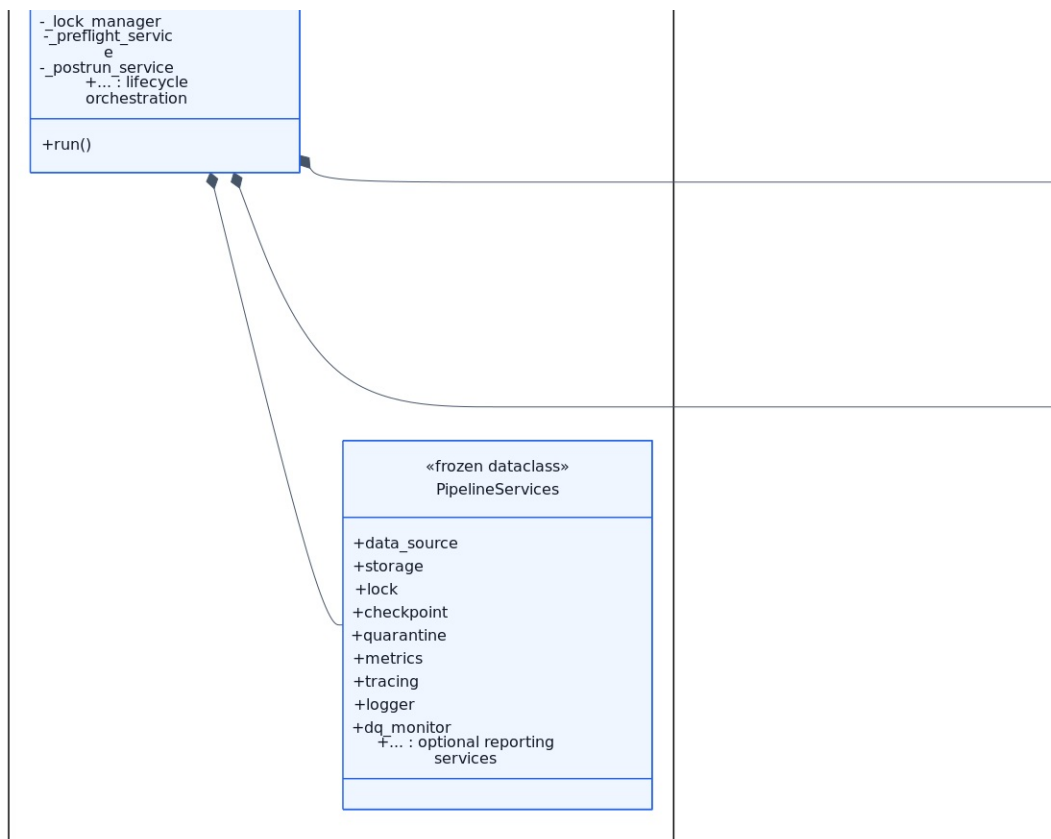
06-config-classes

Диаграмма Class Diagram: Configuration Classes из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 06-config-classes. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Domain and application configuration hierarchy.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: RuntimeConfig, PipelineConfig, TableConfig, DQConfig, SilverFilterConfig, GoldFilterConfig. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

07-application-core-services — Application Core Services



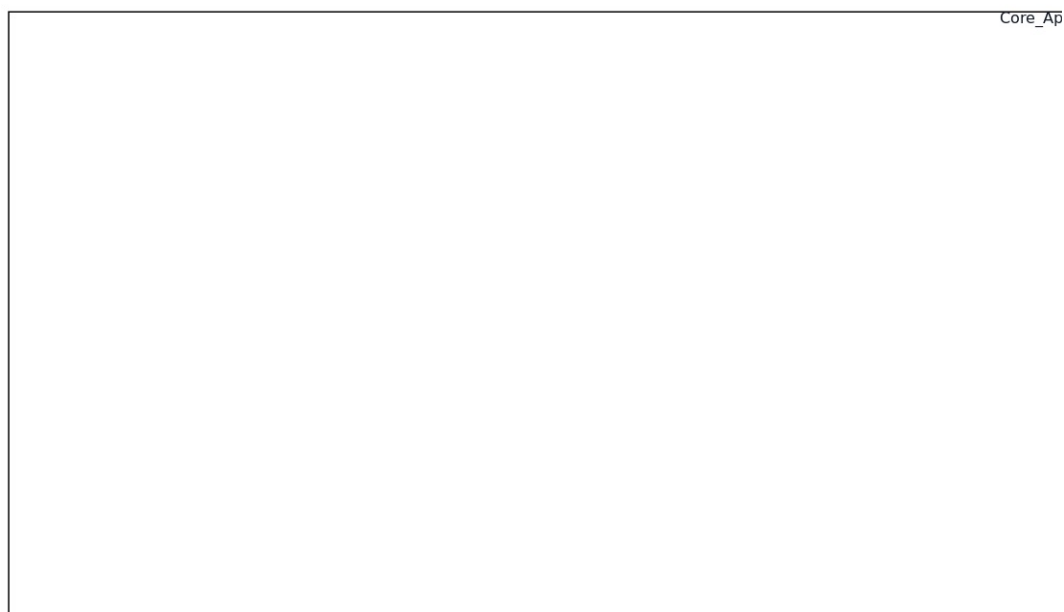


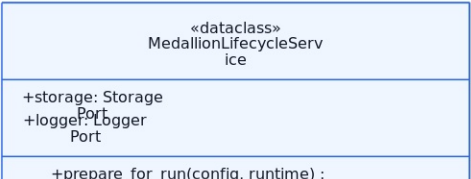


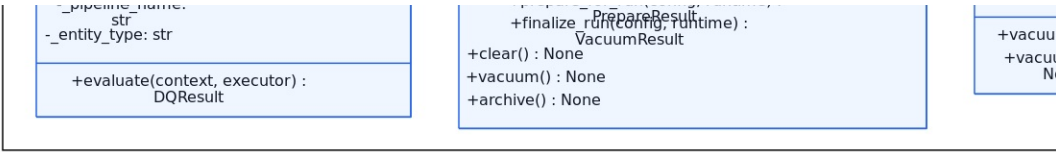
07-application-core-services

Диаграмма Class Diagram: Application Core Services из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 07-application-core-services. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: PipelineRunner, BatchExecutor, and their composition.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Ключевые пространства имен/секции включают: Runner_Core, Batch_Processing, Execution_Managers, Support_Services. Такое разбиение показывает логические подсистемы и границы взаимодействия между ними. Показательные классы этой диаграммы: PipelineRunner, PipelineServices, BatchExecutor, BatchTransformer, BatchWriter, BatchMemoryManager. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

08-application-services — Application Services



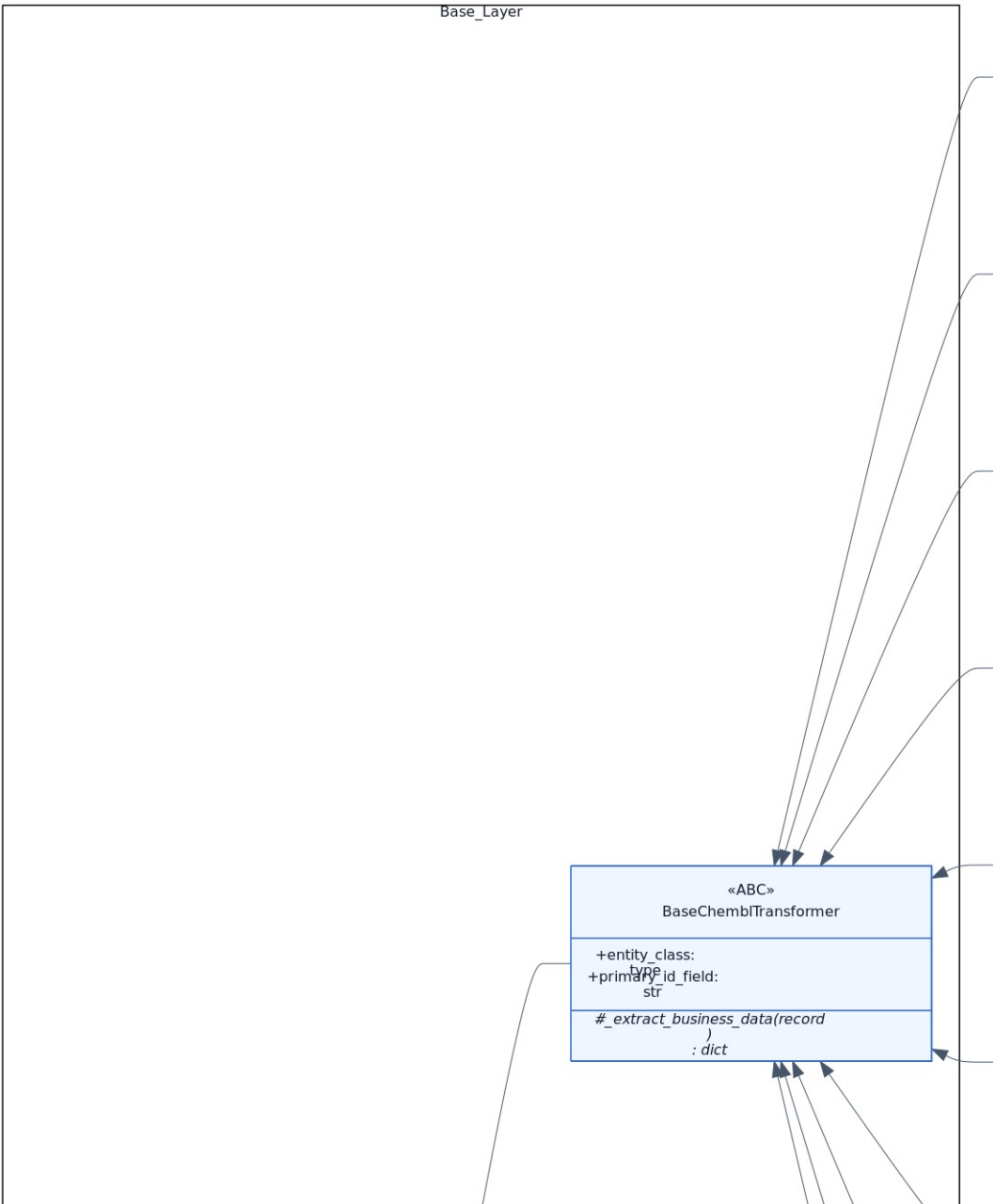


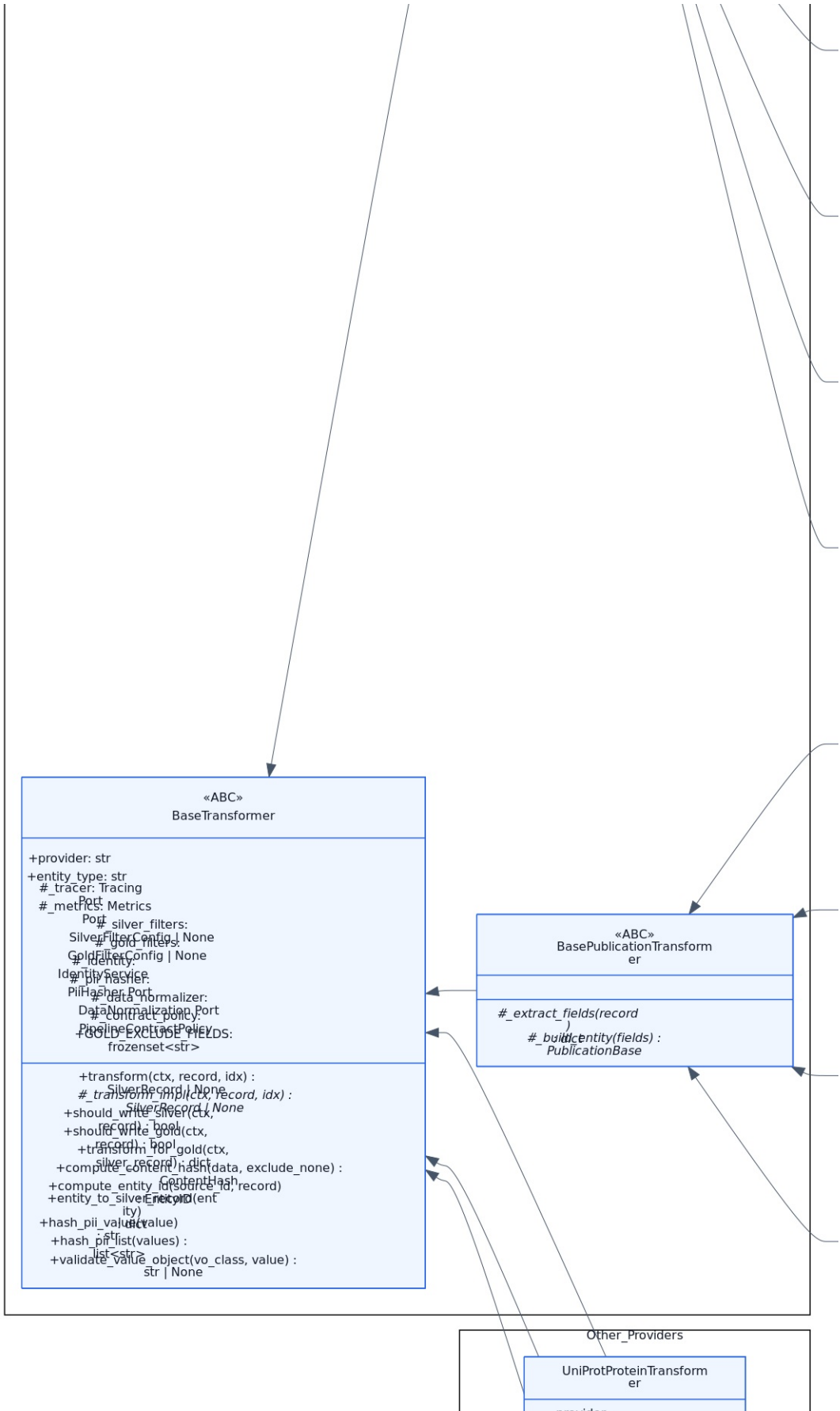


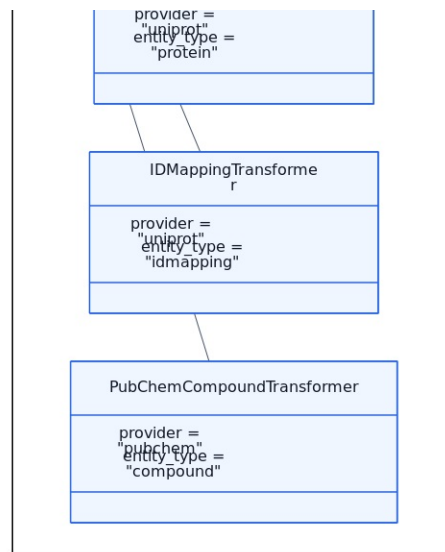
08-application-services

Диаграмма Class Diagram: Application Services из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 08-application-services. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: High-level application services.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Ключевые пространства имен/секции включают: Core_Application, Operational_Services, DQ_Analyzers. Такое разбиение показывает логические подсистемы и границы взаимодействия между ними. Показательные классы этой диаграммы: DataQualityService, DQReportService, MedallionLifecycleService, VacuumService, PipelineObserver, LifecyclePhase. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

09-transformers — Transformers



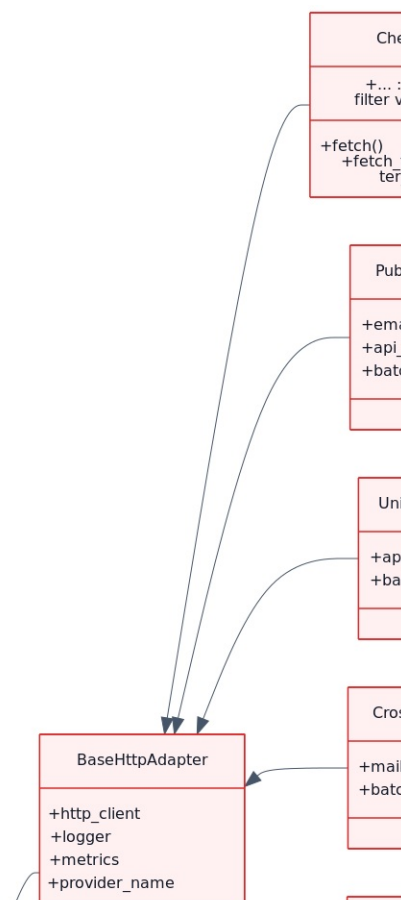


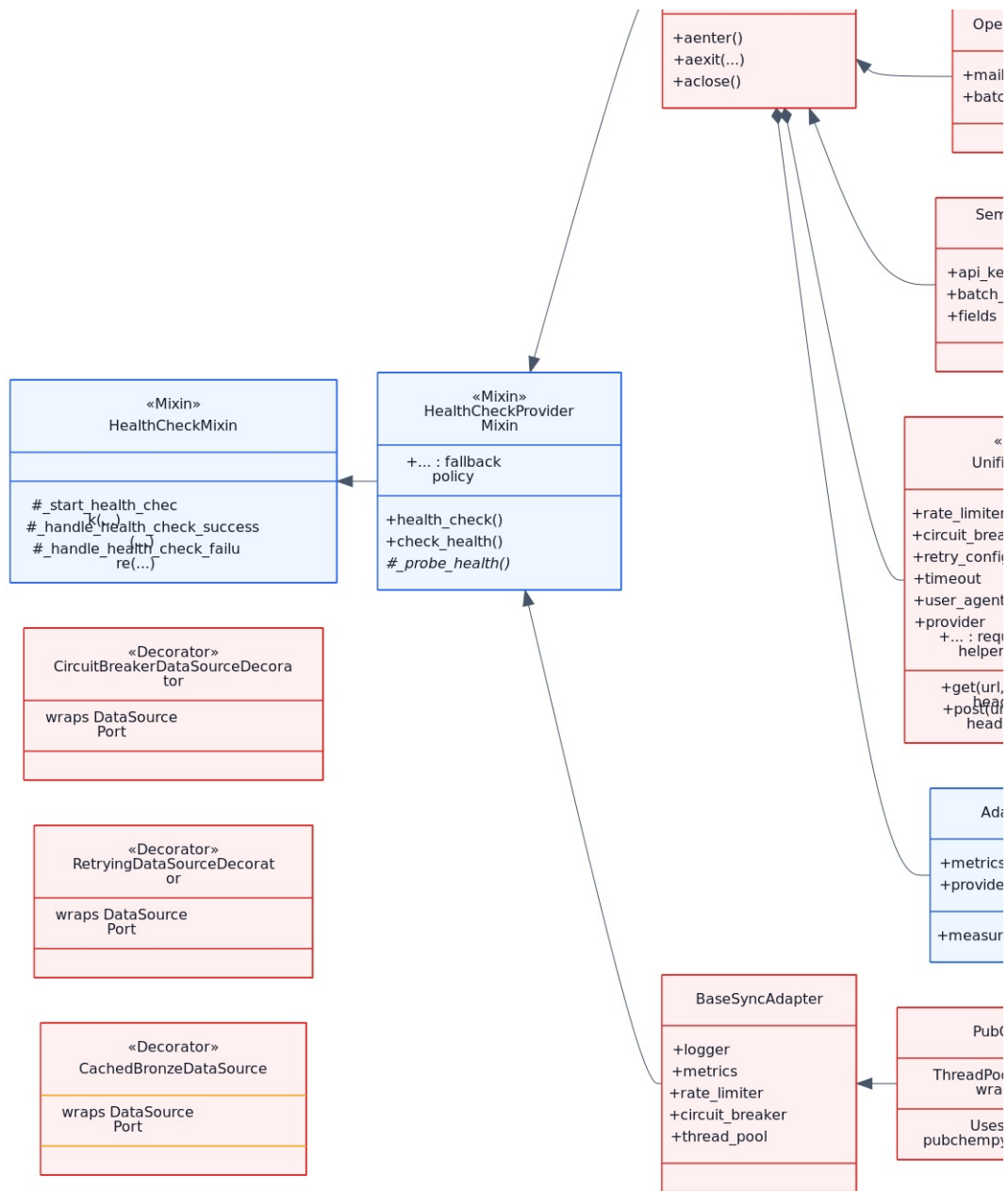


09-transformers

Диаграмма Class Diagram: Transformers из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 09-transformers. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: BaseTransformer hierarchy and provider-specific implementations.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Ключевые пространства имен/секции включают: Base_Layer, ChEMBL_Transformers, Publication_Enrichers, Other_Providers. Такое разбиение показывает логические подсистемы и границы взаимодействия между ними. Показательные классы этой диаграммы: BaseTransformer, BaseChEMBLTransformer, BasePublicationTransformer, ActivityTransformer, AssayTransformer, MoleculeTransformer. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

10-adapters — Adapters

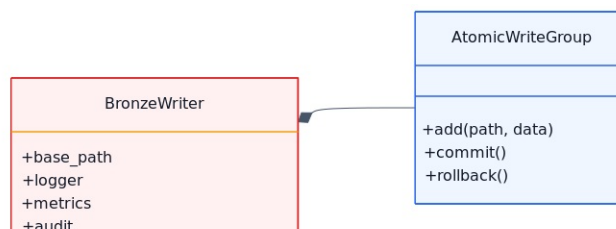




10-adapters

Диаграмма Class Diagram: Infrastructure Adapters из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 10-adapters. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях источника отмечен фокус диаграммы: HTTP adapter class hierarchy with mixins.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: HealthCheckMixin, HealthCheckProviderMixin, BaseHttpAdapter, BaseSyncAdapter, UnifiedHTTPClient, ChEMBLAdapter. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

11-storage — Storage



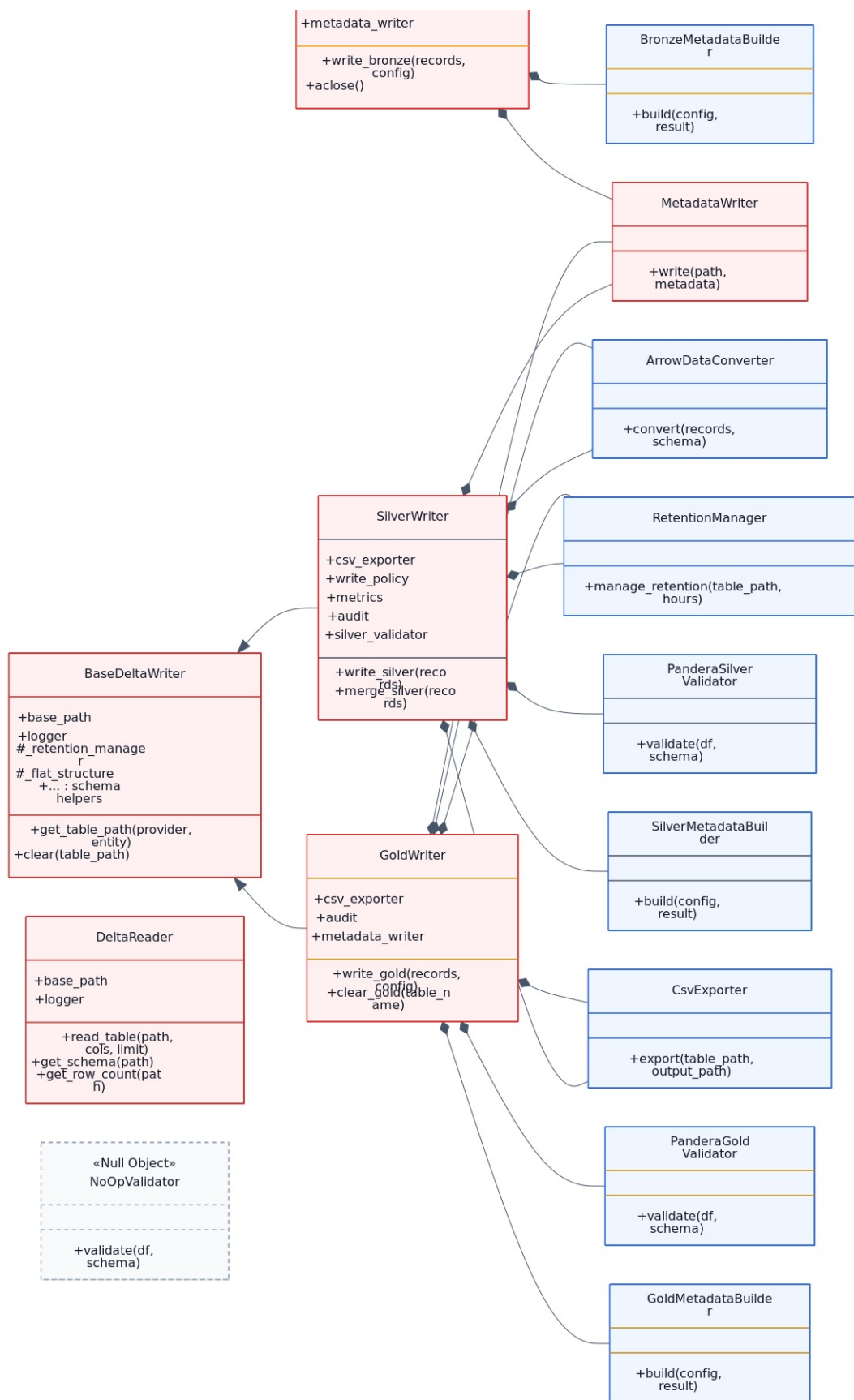
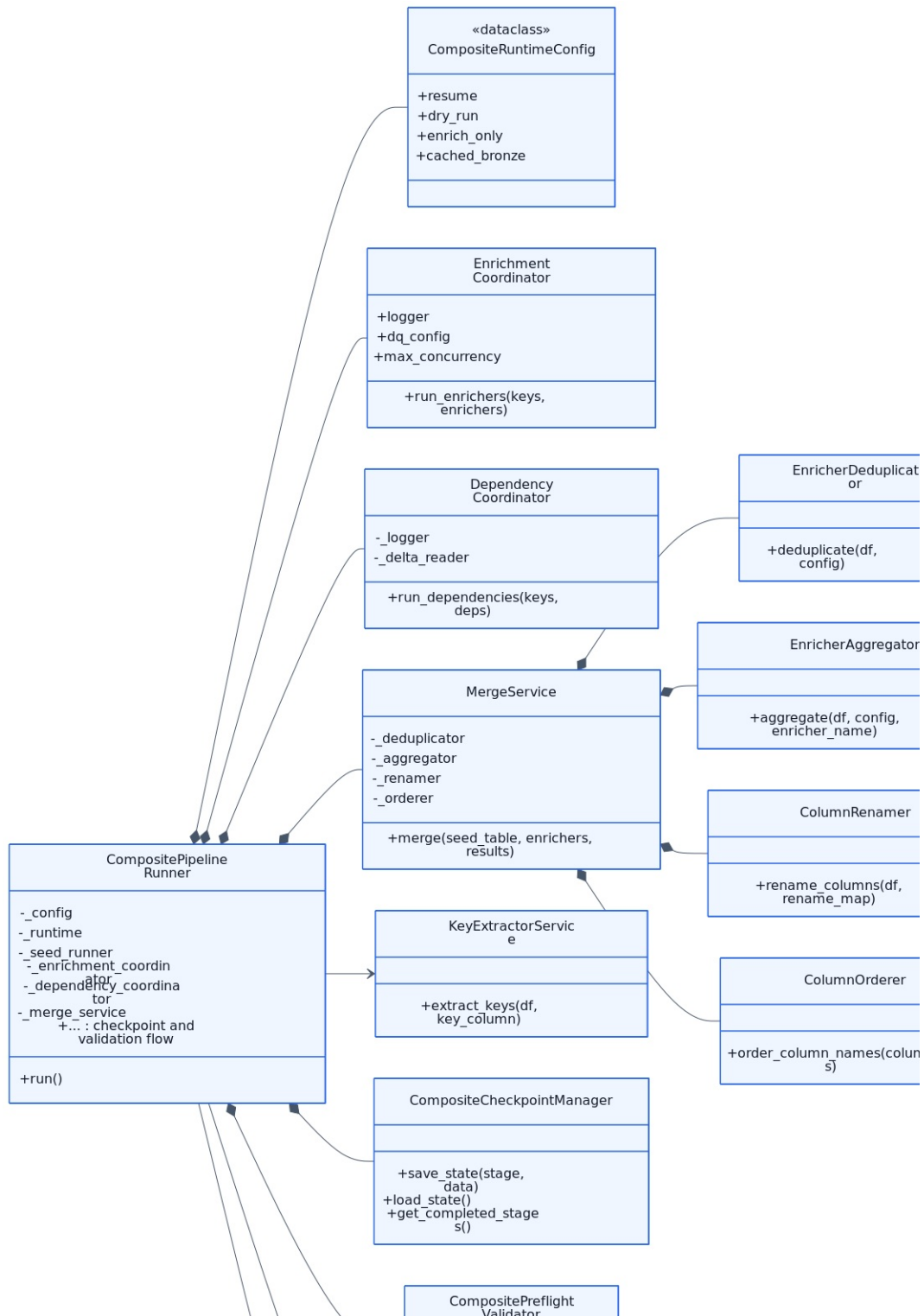
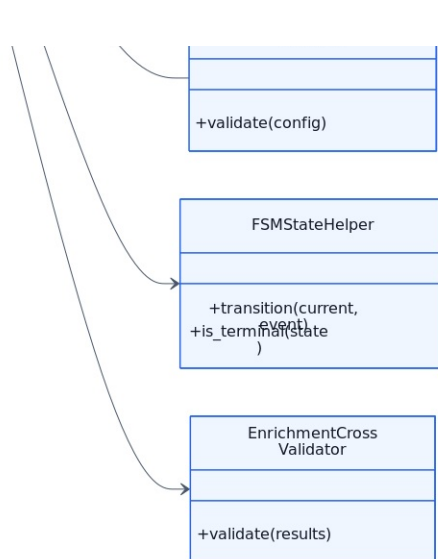


Диаграмма Class Diagram: Storage Components из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 11-storage. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях источника отмечен фокус диаграммы: Bronze/Silver/Gold writers and supporting classes.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: BaseDeltaWriter, BronzeWriter, SilverWriter, GoldWriter, DeltaReader, ArrowDataConverter. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

12-composite-pipeline — Composite Pipeline

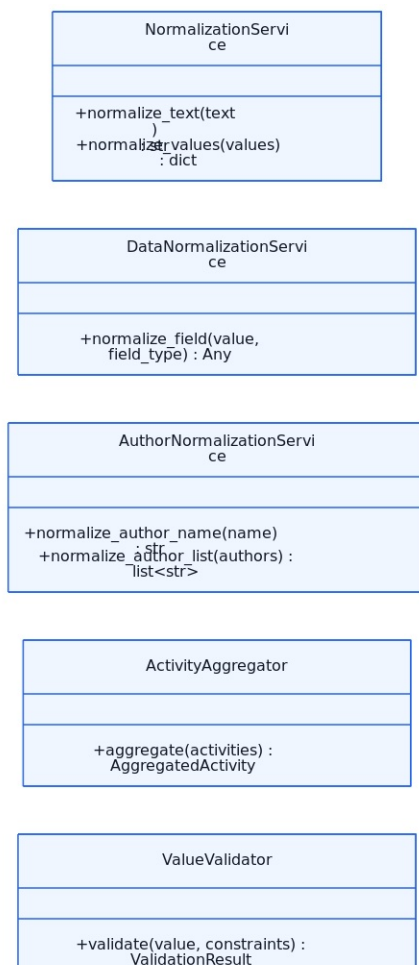


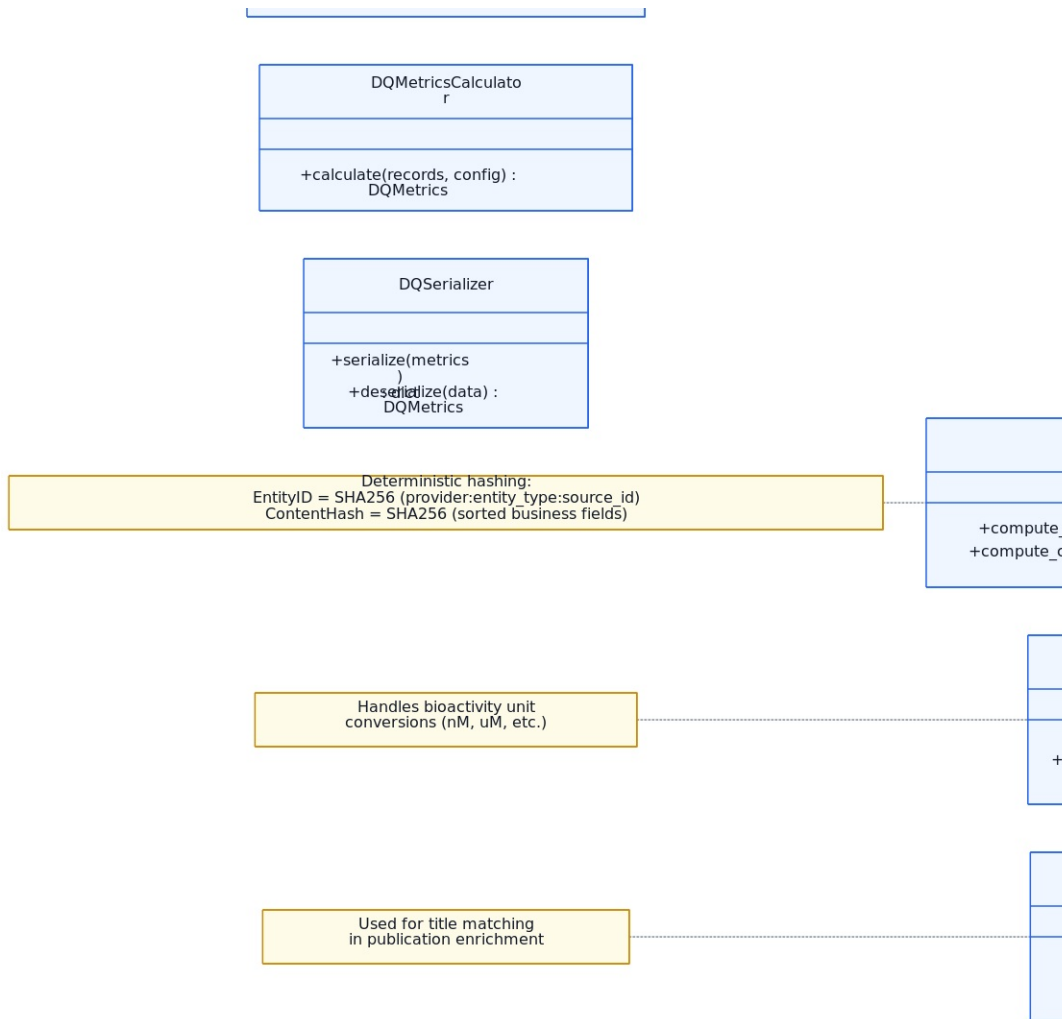


12-composite-pipeline

Диаграмма Class Diagram: Composite Pipeline Components из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 12-composite-pipeline. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Runner, coordinators, merge service, and FSM.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: CompositePipelineRunner, CompositeRuntimeConfig, EnrichmentCoordinator, DependencyCoordinator, MergeService, EnricherAggregator. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

13-domain-services — Domain Services

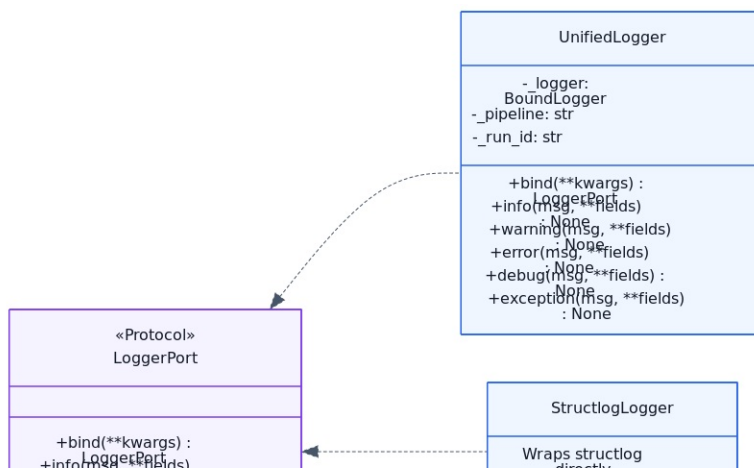


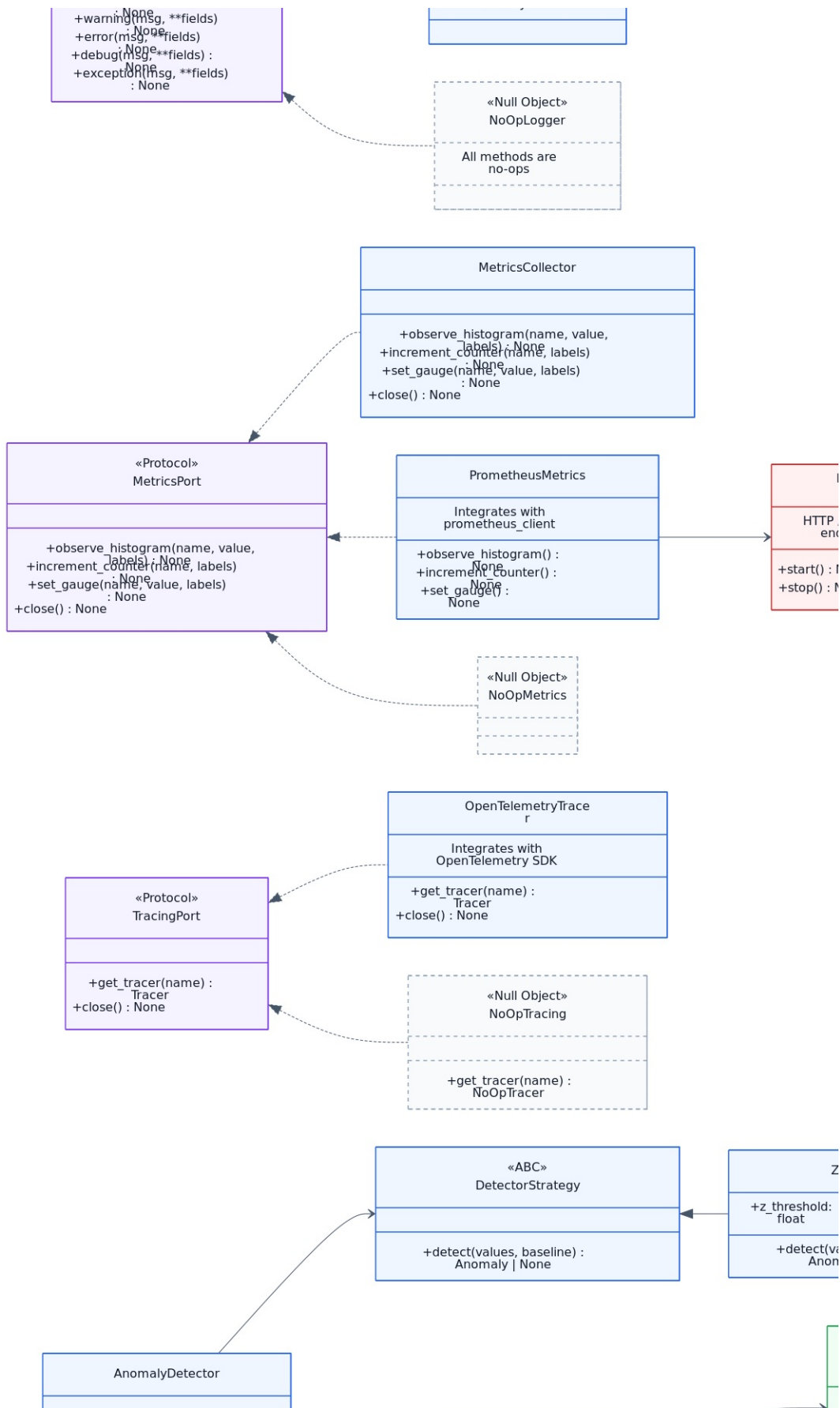


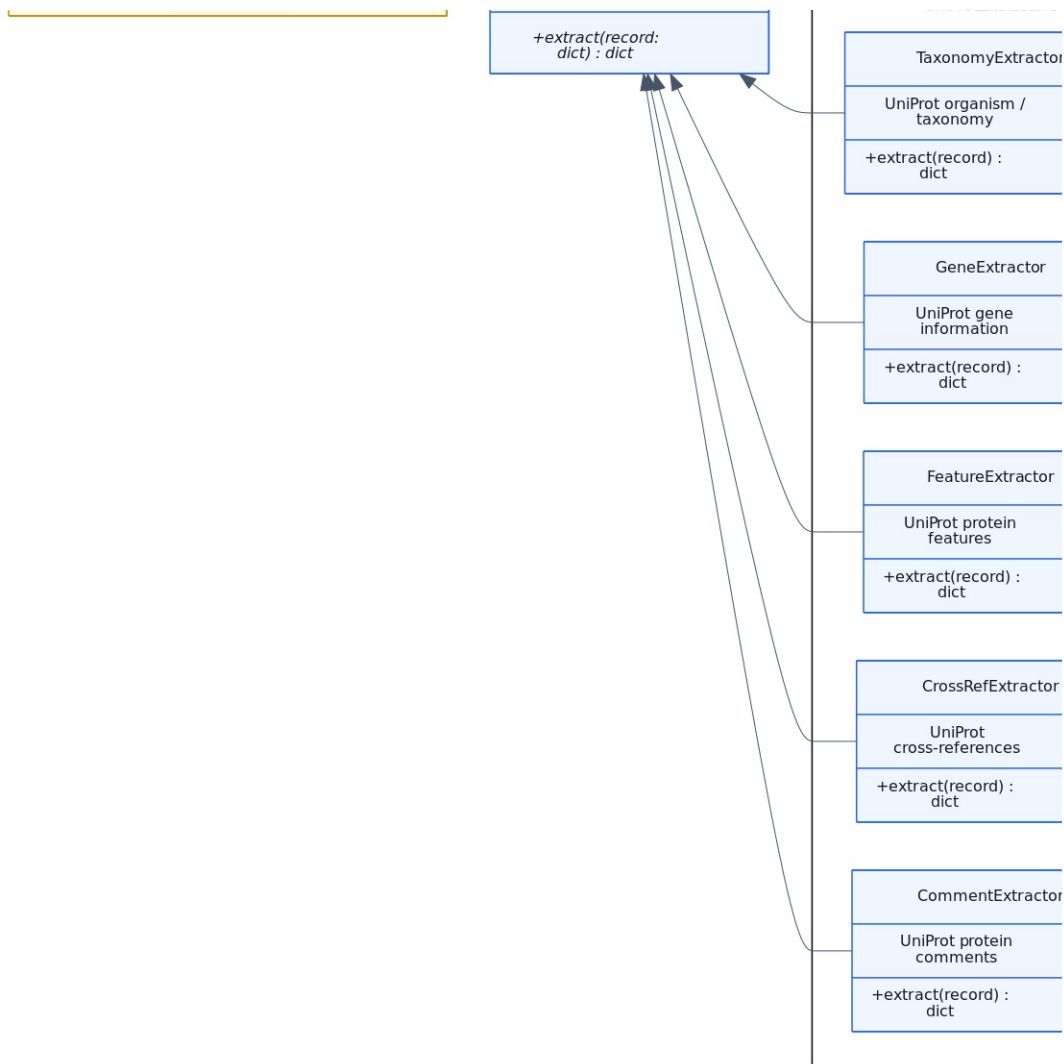
13-domain-services

Диаграмма Class Diagram: Domain Services из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 13-domain-services. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Pure domain services without I/O.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: IdentityService, NormalizationService, DataNormalizationService, AuthorNormalizationService, ActivityAggregator, UnitConverter. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

14-observability — Observability



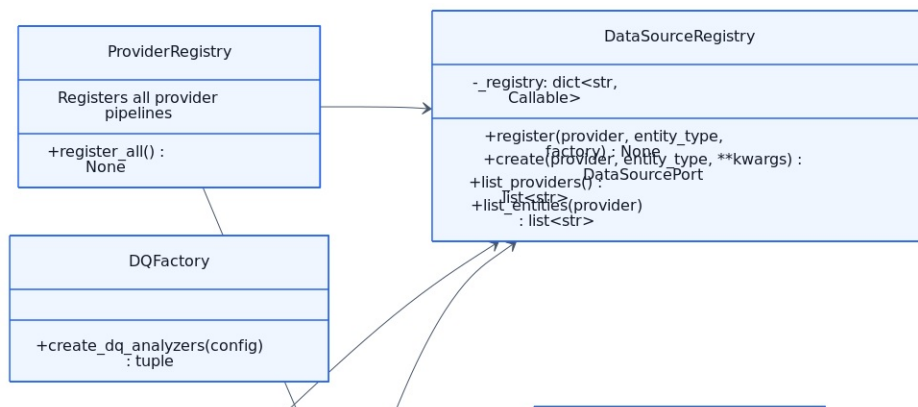


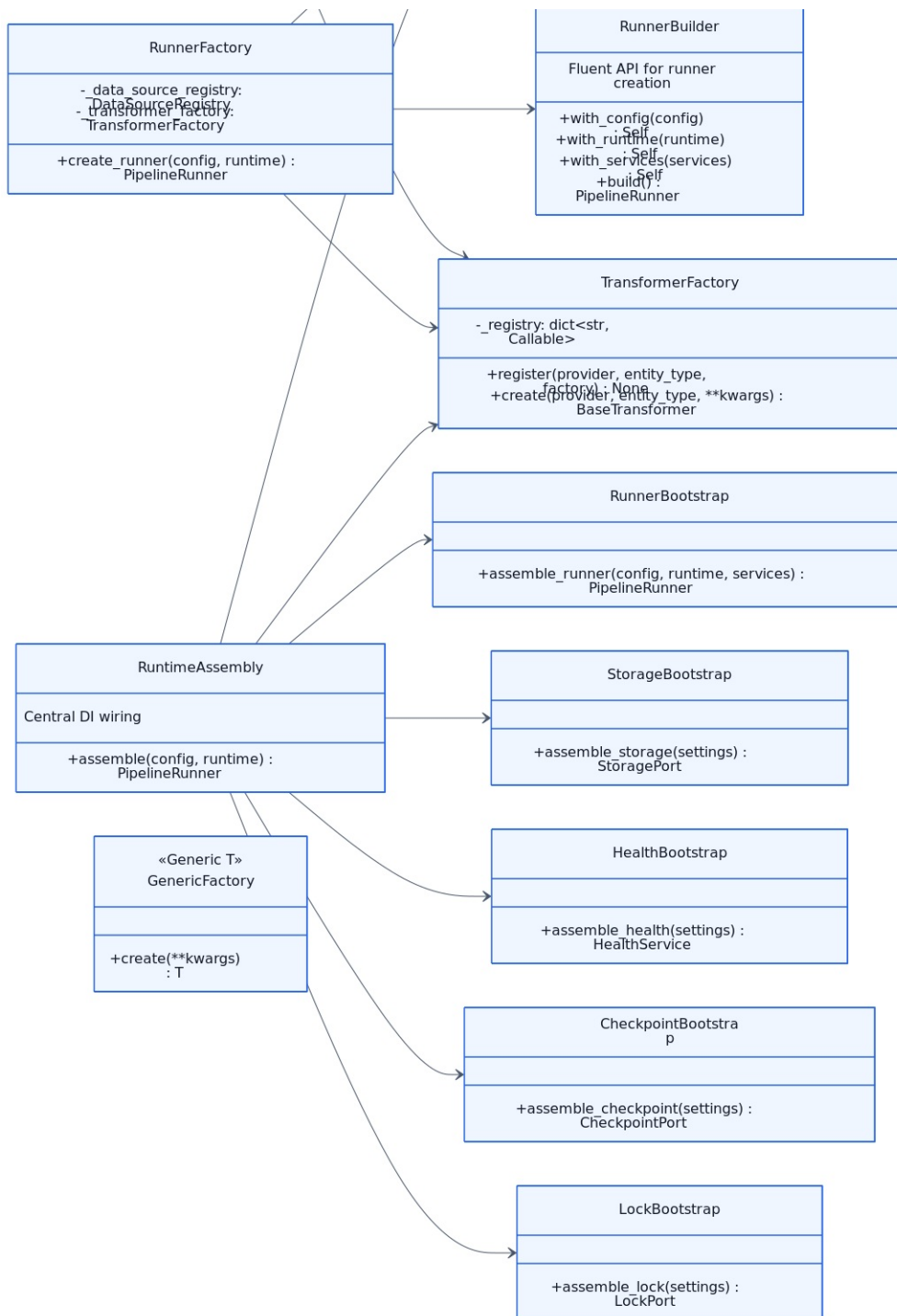


15-extractors

Диаграмма Class Diagram: Field Extractors из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 15-extractors. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях источника отмечен фокус диаграммы: Extractor pattern used in transformers.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Ключевые пространства имен/секции включают: PubMedExtractors, UniProtExtractors. Такое разбиение показывает логические подсистемы и границы взаимодействия между ними. Показательные классы этой диаграммы: BaseFieldExtractor, AbstractExtractor, AuthorExtractor, DateExtractor, ClassificationExtractor, IdentifierExtractor. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.

16-factories-bootstrap — Factories Bootstrap





16-factories-bootstrap

Диаграмма Class Diagram: Factories & Bootstrap из семейства class-diagrams описывает структуру ключевых классов, интерфейсов и их связей внутри BioETL. Формат classDiagram помогает быстро понять распределение ответственности между компонентами и стабильные точки расширения в модуле 16-factories-bootstrap. Уровень детализации указан как Class / Interface, поэтому схема полезна для ревью архитектуры, валидации зависимостей и оценки влияния рефакторинга на API классов, сервисов и фабрик. В комментариях исходника отмечен фокус диаграммы: Composition layer factories and DI assembly.. Это делает интерпретацию схемы однозначной и снижает риск расхождений между документацией, тестами и реальной реализацией. Показательные классы этой диаграммы: DataSourceRegistry, TransformerFactory, RunnerFactory, DQFactory, RuntimeAssembly, RunnerBootstrap. По ним удобно проверять контракты, наследование и композицию, а также согласованность терминов в кодовой базе. В метаданных также отражена оценка плотности (@nodes=n/a), что помогает контролировать читаемость диаграммы и вовремя делить её на более узкие представления при росте сложности.